PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO GUÍA DE APRENDIZAJE – ACTIVIDADES

Actividad: Resolver problemas mediante algoritmos computacionales que incluyan procedimientos y funciones en pseint

Al finalizar esta guía, el aprendiz será capaz de:

- Comprender la diferencia entre procedimientos y funciones.
- Aplicar procedimientos para organizar tareas.
- Crear funciones para resolver problemas que requieren retornar valores.
- Diseñar soluciones modulares usando subprogramas en PSeInt.

Conceptos teóricos

- **Subprogramas**

Un subprograma es un bloque de código que realiza una tarea específica. En PSeInt existen dos tipos principales:

Tipo	Devuelve valor	Uso principal
Procedimiento	❌ No	Ejecuta acciones
Función	✅ Sí	Calcula y retorna un valor

Procedimientos

- No devuelven un valor.
- Se utilizan para mostrar información, realizar procesos repetitivos o modificar variables.

Estructura en PSeInt:

```
SubProceso nombreProcedimiento(parametros)
    acciones
FinSubProceso
```

Funciones

- Devuelven un valor.
- Se usan para realizar cálculos o tomar decisiones.

Estructura en PSeInt:

```
Funcion retorno <- nombreFuncion(parametros)
    acciones
    retorno <- resultado
FinFuncion
```

Ejercicios ejemplo

Ejemplo de Procedimiento: Saludo al usuario

```
SubProceso Saludar(nombre)
    Escribir "Hola ", nombre, ", bienvenido al sistema"
FinSubProceso

Proceso Principal
    Definir nombre Como Cadena
    Escribir "Digite su nombre: "
    Leer nombre
    Saludar(nombre)
FinProceso
```

Ejemplo de Procedimiento: Calcular el área de un rectángulo

```
SubProceso AreaRectangulo(base, altura)
    Definir area Como Real
    area <- base * altura
    Escribir "El área del rectángulo es: ", area
FinSubProceso

Proceso Principal
    AreaRectangulo(5, 3)
FinProceso
```

Ejemplo de Función: Sumar dos números

```
Funcion resultado <- Sumar(a, b)
    resultado <- a + b
FinFuncion

Proceso Principal
    Definir res Como Entero
    res <- Sumar(10, 5)
    Escribir "La suma es: ", res
FinProceso
```

Ejemplo de Función: Determinar si un número es par

```
Funcion esPar <- EsPar(num)
    Si num % 2 = 0 Entonces
        esPar <- Verdadero
    SiNo
        esPar <- Falso
    FinSi
FinFuncion

Proceso Principal
    Definir n Como Entero

    Escribir "Digite un número:"
    Leer n

    Si EsPar(n) Entonces
        Escribir "El número es par"
    SiNo
        Escribir "El número es impar"
    FinSi
FinProceso
```

Ejercicios propuestos

1. Crear un procedimiento que muestre un menú con 3 opciones.
2. Crear un procedimiento que reciba un nombre y una edad y los muestre.
3. Crear una función que reciba dos números y devuelva el mayor.
4. Crear una función que convierta kilómetros a metros.
5. Crear un procedimiento que imprima la tabla de multiplicar de un número.
6. Crear una función que convierta grados Celsius a Fahrenheit.
7. Crear un procedimiento que reciba 3 notas y muestre el promedio.
8. Crear una función que calcule el área de un círculo.
9. Crear un procedimiento que reciba una palabra y la muestre 5 veces.
10. Crear una función que determine si un número es positivo, negativo o cero.
11. Crear una función que calcule el factorial de un número.
12. Crear un procedimiento que muestre los números primos del 1 al 100.
13. Crear una función que valide si una contraseña tiene más de 8 caracteres.
14. Crear un procedimiento que ordene 5 números de menor a mayor.
15. Crear una función que calcule el promedio de un arreglo de 10 posiciones.

Actividad para entregar como evidencia:

Situación problema: Diseñar un algoritmo que simule un sistema de notas utilizando funciones y procedimientos. Simular las notas obtenidas en la materia de programación. La cantidad total de notas por cada alumno es de 3.

Debe incluir:

Procedimiento: Mostrar menú

Procedimiento: Leer las notas de 5 alumnos
Función: Calcular promedio
Función: Determinar si ganó o perdió (promedio ≥ 3)
Procedimiento: Mostrar resultados

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	03-12-2025